assignment 3

2026-05-10

```r
library(DBI)
```

```
## Warning: package 'DBI' was built under R version 4.4.3
```

```r
library(RSQLite)
```

```
## Warning: package 'RSQLite' was built under R version 4.4.3
```

```r
db_path <- "Stats.StackExchange.db"
con <- dbConnect(SQLite(), dbname = db_path)
```

```
SELECT name FROM sqlite_master WHERE type='table';
```

Table 1: Displaying records 1 - 10

name
Badges
Comments
PostHistory
PostLinks
Posts
Users
Votes
TagPosts
PostTypeIdMap
BadgeClassMap

```
PRAGMA table_info(PostTypeIdMap);
```

Table 2: 2 records

cid	name	type	notnull	dflt_value	pk
0	id	INTEGER	0	NA	0
1	value	TEXT	0	NA	0

#1. What is the date range for the questions and answers in this database?

```sql
SELECT
  CASE PostTypeId WHEN 1 THEN 'Question' WHEN 2 THEN 'Answer' END AS PostType,
  MIN(CreationDate) AS earliest,
  MAX(CreationDate) AS latest,
  COUNT(*) AS n
FROM Posts
WHERE PostTypeId IN (1, 2)
GROUP BY PostTypeId

UNION ALL

SELECT 'Both', MIN(CreationDate), MAX(CreationDate), COUNT(*)
FROM Posts
WHERE PostTypeId IN (1, 2);
```

```r
q1
```

```
##   PostType                earliest                  latest      n
## 1 Question 2009-02-02T14:21:12.103 2023-03-05T05:10:18.393 204370
## 2   Answer 2009-02-02T14:24:31.740 2023-03-05T04:48:34.853 197928
## 3     Both 2009-02-02T14:21:12.103 2023-03-05T05:10:18.393 402298
```

#question 2

```sql
SELECT COUNT(*) AS n_users FROM Users;
```

```sql
SELECT COUNT(*) AS n_users_since_2020
FROM Users
WHERE CreationDate >= '2020-01-01';
```

```sql
SELECT
  CAST(strftime('%Y', CreationDate) AS INTEGER) AS year,
  COUNT(*) AS n_users
FROM Users
WHERE CreationDate IS NOT NULL
GROUP BY year
ORDER BY year;
```

```r
q2_total
```

```
##   n_users
## 1  321677
```

```r
q2_since2020
```

```
##   n_users_since_2020
## 1             100796
```

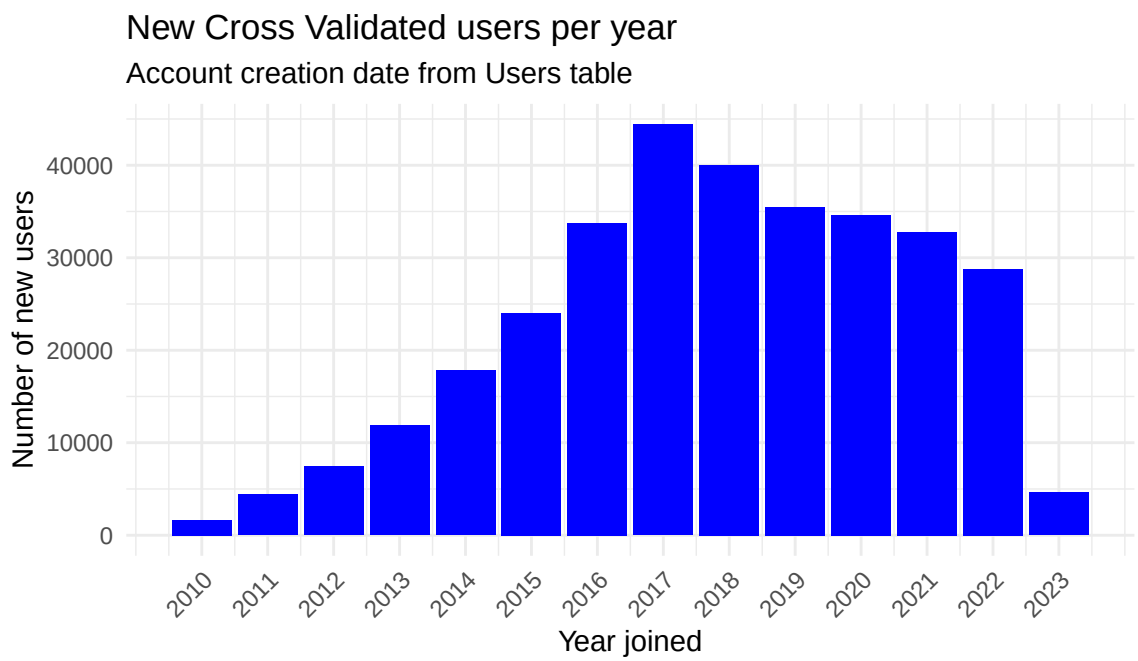
```r
q2_by_year
```

```
##    year n_users
## 1  2010    1668
## 2  2011    4396
## 3  2012    7450
## 4  2013   11846
## 5  2014   17809
## 6  2015   24012
## 7  2016   33753
## 8  2017   44416
## 9  2018   40040
## 10 2019   35491
## 11 2020   34617
## 12 2021   32765
## 13 2022   28801
## 14 2023    4613
```

```r
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.2
```

```r
ggplot(q2_by_year, aes(x = year, y = n_users)) +
  geom_col(fill = "blue") +
  scale_x_continuous(breaks = q2_by_year$year) +
  labs(
    title = "New Cross Validated users per year",
    subtitle = "Account creation date from Users table",
    x = "Year joined",
    y = "Number of new users"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



#question 3

```
SELECT
  u.Id AS user_id,
  u.DisplayName AS display_name,
  u.Reputation AS reputation,
  COUNT(*) AS n_answers
FROM Posts p
JOIN Users u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 2
  AND p.OwnerUserId IS NOT NULL
  AND p.OwnerUserId != -1
GROUP BY u.Id
ORDER BY n_answers DESC
LIMIT 10;
```

```
q3
```

```
##    user_id       display_name reputation n_answers
## 1      805             Glen_b     268966      4733
## 2      919             whuber     304878      3051
## 3      686         Peter Flom      97091      2875
## 4    28500                EdM      76431      2703
## 5    35989                Tim     126510      2454
## 6    11887 kjetil b halvorsen      70519      2371
## 7   173082                Ben     108435      2240
## 8    85665            BruceET      53074      2189
## 9     1352    Stephan Kolassa     107905      2124
## 10  204068              gunes      54443      1873
```

#question 4

```sql
WITH
q AS (
  SELECT OwnerUserId AS uid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 1 AND OwnerUserId IS NOT NULL AND OwnerUserId != -1
  GROUP BY OwnerUserId
),
a AS (
  SELECT OwnerUserId AS uid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 2 AND OwnerUserId IS NOT NULL AND OwnerUserId != -1
  GROUP BY OwnerUserId
),
c AS (
  SELECT UserId AS uid, COUNT(*) AS n
  FROM Comments
  WHERE UserId IS NOT NULL AND UserId != -1
  GROUP BY UserId
),
flags AS (
  SELECT
    u.Id AS uid,
    CASE WHEN q.n > 0 THEN 1 ELSE 0 END AS asked,
    CASE WHEN a.n > 0 THEN 1 ELSE 0 END AS answered,
    CASE WHEN c.n > 0 THEN 1 ELSE 0 END AS commented
  FROM Users u
  LEFT JOIN q ON q.uid = u.Id
  LEFT JOIN a ON a.uid = u.Id
  LEFT JOIN c ON c.uid = u.Id
)
SELECT
  SUM(asked) AS asked_at_least_one,
  SUM(CASE WHEN answered = 1 AND asked = 0 AND commented = 0 THEN 1 ELSE 0 END) AS only_answered,
  SUM(CASE WHEN commented = 1 AND asked = 0 AND answered = 0 THEN 1 ELSE 0 END) AS only_commented,
  SUM(CASE WHEN asked = 0 AND answered = 0 AND commented = 0 THEN 1 ELSE 0 END) AS no_posts,
  COUNT(*) AS total_users
FROM flags;
```

```sql
q4
```

```
##   asked_at_least_one only_answered only_commented no_posts total_users
## 1              88923         10263           5009   213257      321677
```

#question 5

```sql
SELECT
  p.PostTypeId,
  pt.value AS post_type,
  COUNT(*) AS n_posts
FROM Posts p
LEFT JOIN PostTypeIdMap pt ON pt.id = p.PostTypeId
GROUP BY p.PostTypeId
ORDER BY n_posts DESC;
```

```r
q5
```

```
##   PostTypeId                                                  post_type
## 1          1                                                   Question
## 2          2                                                     Answer
## 3          5                                                   Tag wiki
## 4          4                                          Tag wiki excerpt
## 5          6                                       Moderator nomination
## 6          3                                          Orphaned tag wiki
## 7          7 Wiki placeholder (seems to only be the election description)
##   n_posts
## 1  204370
## 2  197928
## 3    1444
## 4    1444
## 5      23
## 6       6
## 7       5
```

```r
nrow(q5)
```

```
## [1] 7
```

#question 6

```sql
WITH questions AS (
  SELECT Id, CreationDate, AcceptedAnswerId
  FROM Posts
  WHERE PostTypeId = 1
),
answer_stats AS (
  SELECT
    ParentId AS qid,
    MIN(CreationDate) AS first_answer_date,
    MAX(CreationDate) AS last_answer_date
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
),
accepted AS (
  SELECT Id AS aid, CreationDate AS accepted_date
  FROM Posts
  WHERE PostTypeId = 2
)
SELECT
  (julianday(a.first_answer_date) - julianday(q.CreationDate)) * 24 AS hours_to_first,
  (julianday(acc.accepted_date)   - julianday(q.CreationDate)) * 24 AS hours_to_accepted,
  (julianday(a.last_answer_date)  - julianday(q.CreationDate)) * 24 AS hours_to_last
FROM questions q
LEFT JOIN answer_stats a   ON a.qid = q.Id
LEFT JOIN accepted    acc  ON acc.aid = q.AcceptedAnswerId;
```

```r
summary(q6_times)
```

```
##  hours_to_first      hours_to_accepted  hours_to_last
##  Min.   :-14964.90   Min.   :   -1.76   Min.   :    -7.09
##  1st Qu.:     0.77   1st Qu.:    0.86   1st Qu.:     1.32
##  Median :     3.11   Median :    3.39   Median :     7.37
##  Mean   :  1306.30   Mean   :  607.35   Mean   :  3767.99
##  3rd Qu.:    25.36   3rd Qu.:   22.06   3rd Qu.:   118.08
##  Max.   :100732.06   Max.   :93053.85   Max.   :109884.72
##  NA's   :66970       NA's   :136366     NA's   :66970
```

```r
sapply(q6_times, median, na.rm = TRUE)
```

```
##    hours_to_first hours_to_accepted     hours_to_last
##          3.109062          3.385504          7.368921
```

```r
sapply(q6_times, mean,   na.rm = TRUE)
```

```
##    hours_to_first hours_to_accepted     hours_to_last
##         1306.3042          607.3489         3767.9928
```

```r
WITH questions AS (
  SELECT Id FROM Posts WHERE PostTypeId = 1
),
answered AS (
  SELECT DISTINCT ParentId AS qid FROM Posts WHERE PostTypeId = 2
)
SELECT
  (SELECT COUNT(*) FROM questions) AS total_questions,
  (SELECT COUNT(*) FROM questions q LEFT JOIN answered a ON a.qid = q.Id
     WHERE a.qid IS NULL) AS questions_no_response,
  (SELECT COUNT(*) FROM Posts) AS total_posts;
```

```r
q6_noresponse
```

```
##   total_questions questions_no_response total_posts
## 1          204370                 66970      405220
```

```r
q6_noresponse$pct_of_all_posts <- 100 * q6_noresponse$questions_no_response / q6_noresponse$total_posts
q6_noresponse$pct_of_questions <- 100 * q6_noresponse$questions_no_response / q6_noresponse$total_questi
q6_noresponse
```

```
##   total_questions questions_no_response total_posts pct_of_all_posts
## 1          204370                 66970      405220         16.52682
##   pct_of_questions
## 1           32.769
```

#question 7

```sql
SELECT
  tp.Tag AS tag,
  COUNT(*) AS n_questions
FROM TagPosts tp
JOIN Posts p ON p.Id = tp.Id
WHERE p.PostTypeId = 1
GROUP BY tp.Tag
ORDER BY n_questions DESC
LIMIT 50;
```

```
q7
```

```
##                          tag n_questions
## 1                          r       28495
## 2                 regression       28146
## 3           machine-learning       19355
## 4                time-series       13745
## 5                probability       11894
## 6         hypothesis-testing       10091
## 7              distributions        9147
## 8                 self-study        7985
## 9            neural-networks        7793
## 10                  bayesian        7628
## 11                  logistic        7507
## 12  mathematical-statistics        7455
## 13            classification        6654
## 14               correlation        6074
## 15 statistical-significance        6038
## 16       normal-distribution        5877
## 17               mixed-model        5837
## 18       multiple-regression        5265
## 19                     anova        5100
## 20                    python        4605
## 21       confidence-interval        4367
## 22 generalized-linear-model        4276
## 23                  variance        4042
## 24                clustering        3932
## 25               forecasting        3726
## 26                    t-test        3486
## 27          categorical-data        3468
## 28          cross-validation        3385
## 29                       pca        3333
## 30        maximum-likelihood        3209
## 31                estimation        3159
## 32                 lme4-nlme        3156
## 33                  sampling        3104
## 34         predictive-models        2971
## 35                  survival        2960
## 36        data-visualization        2949
## 37                 inference        2899
## 38                     arima        2818
## 39                   p-value        2709
## 40                      mean        2705
```

```
## 41             optimization        2688
## 42            least-squares        2682
## 43        repeated-measures        2612
## 44         chi-squared-test        2559
## 45                 modeling        2501
## 46               references        2451
## 47    multivariate-analysis        2430
## 48             econometrics        2422
## 49              interaction        2421
## 50             linear-model        2412
```

#question 8

```sql
WITH questions AS (
  SELECT Id FROM Posts WHERE PostTypeId = 1
),
answer_counts AS (
  SELECT ParentId AS qid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
),
comment_counts AS (
  SELECT PostId AS qid, COUNT(*) AS n
  FROM Comments
  GROUP BY PostId
)
SELECT
  q.Id AS question_id,
  COALESCE(a.n, 0) AS n_answers,
  COALESCE(c.n, 0) AS n_comments
FROM questions q
LEFT JOIN answer_counts  a ON a.qid = q.Id
LEFT JOIN comment_counts c ON c.qid = q.Id;
```

```r
summary(q8[, c("n_answers", "n_comments")])
```

```
##    n_answers          n_comments
##  Min.   :  0.0000   Min.   : 0.000
##  1st Qu.:  0.0000   1st Qu.: 0.000
##  Median :  1.0000   Median : 1.000
##  Mean   :  0.9685   Mean   : 2.171
##  3rd Qu.:  1.0000   3rd Qu.: 3.000
##  Max.   :153.0000   Max.   :54.000
```

```r
table(pmin(q8$n_answers, 10))
```

```
##
##     0     1     2     3     4     5     6     7     8     9    10
## 66970 98602 27191  7246  2408   905   401   210   136    82   219
```

```r
table(pmin(q8$n_comments, 10))
```

```
##
##     0     1     2     3     4     5     6     7     8     9    10
## 76090 30707 30417 20645 14897  9832  6806  4680  3236  2158  4902
```

```r
quantile(q8$n_answers,  probs = c(0.5, 0.75, 0.9, 0.95, 0.99, 1))
```

```
##  50%  75%  90%  95%  99% 100%
##    1    1    2    3    4  153
```

```r
quantile(q8$n_comments, probs = c(0.5, 0.75, 0.9, 0.95, 0.99, 1))
```

```
##  50%  75%  90%  95%  99% 100%
##    1    3    6    8   12   54
```

#question 9

```sql
SELECT
  CAST(strftime('%Y', CreationDate) AS INTEGER) AS year,
  COUNT(*) AS n_questions
FROM Posts
WHERE PostTypeId = 1
  AND CreationDate IS NOT NULL
GROUP BY year
ORDER BY year;
```

```r
q9
```

```
##    year n_questions
## 1  2009           3
## 2  2010        1477
## 3  2011        4935
## 4  2012        8259
## 5  2013       11620
## 6  2014       15888
## 7  2015       18665
## 8  2016       20914
## 9  2017       21437
## 10 2018       19920
## 11 2019       19744
## 12 2020       21108
## 13 2021       17271
## 14 2022       19651
## 15 2023        3478
```

#question 10

```sql
WITH questions AS (
  SELECT
    Id,
    CAST(strftime('%Y', CreationDate) AS INTEGER) AS year
  FROM Posts
  WHERE PostTypeId = 1
    AND CreationDate IS NOT NULL
),
answer_counts AS (
  SELECT ParentId AS qid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
)
SELECT
  q.Id AS question_id,
  q.year,
  1 + COALESCE(a.n, 0) AS posts_per_question
FROM questions q
LEFT JOIN answer_counts a ON a.qid = q.Id;
```

```r
# Summary by year: medians and quartiles
aggregate(posts_per_question ~ year, data = q10,
          FUN = function(x) c(
            median = median(x),
            q75    = quantile(x, 0.75, names = FALSE),
            q95    = quantile(x, 0.95, names = FALSE),
            max    = max(x)
          ))
```

```
##    year posts_per_question.median posts_per_question.q75 posts_per_question.q95
## 1  2009                      8.0                   12.0                   15.2
## 2  2010                      3.0                    5.0                   11.0
## 3  2011                      3.0                    3.0                    6.0
## 4  2012                      2.0                    3.0                    5.0
## 5  2013                      2.0                    3.0                    4.0
## 6  2014                      2.0                    3.0                    4.0
## 7  2015                      2.0                    2.0                    4.0
## 8  2016                      2.0                    2.0                    4.0
## 9  2017                      2.0                    2.0                    3.0
## 10 2018                      2.0                    2.0                    3.0
## 11 2019                      2.0                    2.0                    3.0
## 12 2020                      2.0                    2.0                    3.0
## 13 2021                      2.0                    2.0                    3.0
## 14 2022                      1.0                    2.0                    3.0
## 15 2023                      1.0                    2.0                    3.0
##    posts_per_question.max
## 1                    16.0
## 2                   154.0
## 3                    23.0
## 4                    35.0
## 5                    16.0
## 6                    22.0
```

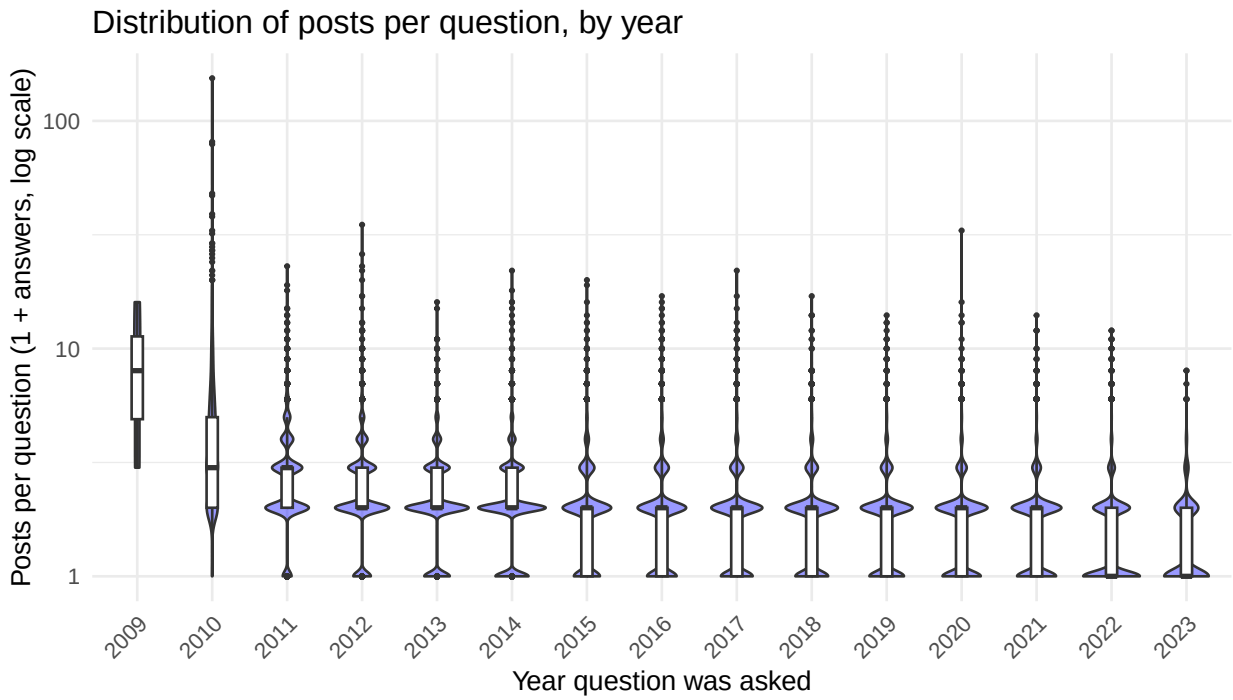
```
## 7                    20.0
## 8                    17.0
## 9                    22.0
## 10                   17.0
## 11                   14.0
## 12                   33.0
## 13                   14.0
## 14                   12.0
## 15                    8.0
```

```r
# Mean alongside the median given the skew
aggregate(posts_per_question ~ year, data = q10, FUN = mean)
```

```
##    year posts_per_question
## 1  2009           9.000000
## 2  2010           4.712255
## 3  2011           2.906991
## 4  2012           2.553094
## 5  2013           2.262134
## 6  2014           2.140609
## 7  2015           2.048486
## 8  2016           1.983886
## 9  2017           1.926529
## 10 2018           1.889809
## 11 2019           1.855602
## 12 2020           1.798560
## 13 2021           1.760292
## 14 2022           1.561905
## 15 2023           1.500000
```

```r
library(ggplot2)

ggplot(q10, aes(x = factor(year), y = posts_per_question)) +
  geom_violin(fill = "blue", alpha = 0.4) +
  geom_boxplot(width = 0.15, outlier.size = 0.5) +
  scale_y_log10() +
  labs(
    title = "Distribution of posts per question, by year",
    x = "Year question was asked",
    y = "Posts per question (1 + answers, log scale)"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



#question 11

```
SELECT
  SUM(CASE WHEN Title LIKE '%neural network%' OR Body LIKE '%neural network%' THEN 1 ELSE 0 END) AS neu
  SUM(CASE WHEN Title LIKE '%classification tree%' OR Body LIKE '%classification tree%' THEN 1 ELSE 0 E
  SUM(CASE WHEN Title LIKE '%anova%' OR Body LIKE '%anova%' THEN 1 ELSE 0 END) AS anova,
  SUM(CASE WHEN Title LIKE '%boosting%' OR Body LIKE '%boosting%' THEN 1 ELSE 0 END) AS boosting,
  SUM(CASE WHEN Title LIKE '%bagging%' OR Body LIKE '%bagging%' THEN 1 ELSE 0 END) AS bagging,
  SUM(CASE WHEN Title LIKE '%clustering%' OR Body LIKE '%clustering%' THEN 1 ELSE 0 END) AS clustering,
  SUM(CASE WHEN Title LIKE '%multiple testing%' OR Body LIKE '%multiple testing%' THEN 1 ELSE 0 END) AS
FROM Posts
WHERE PostTypeId = 1;
```

```
q11
```

```
##   neural_network classification_tree anova boosting bagging clustering
## 1           5742                 164  7431      861     297       4197
##   multiple_testing
## 1              393
```

```
# Reshape into a tall table for nicer presentation
data.frame(
  term = names(q11),
  n_questions = as.integer(q11[1, ])
)
```

```
##                  term n_questions
## 1      neural_network        5742
## 2 classification_tree         164
## 3               anova        7431
```

```
## 4            boosting         861
## 5             bagging         297
## 6          clustering        4197
## 7    multiple_testing         393
```

```
SELECT * FROM LinkTypeMap;
```

Table 3: 2 records

id	value
1	Linked (PostId contains a link to RelatedPostId)
3	Duplicate (PostId is a duplicate of RelatedPostId)

```
WITH answer_counts AS (
  SELECT ParentId AS qid, COUNT(*) AS n_answers
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
),
linked_questions AS (
  SELECT DISTINCT
    pl.RelatedPostId AS qid,
    pl.LinkTypeId
  FROM PostLinks pl
  JOIN Posts p ON p.Id = pl.RelatedPostId
  WHERE p.PostTypeId = 1
)
SELECT
  lq.qid AS question_id,
  lq.LinkTypeId,
  p.OwnerUserId,
  u.Reputation AS asker_reputation,
  COALESCE(a.n_answers, 0) AS n_answers
FROM linked_questions lq
JOIN Posts p ON p.Id  = lq.qid
LEFT JOIN Users u ON u.Id  = p.OwnerUserId
LEFT JOIN answer_counts a ON a.qid = lq.qid;
```

```
q12$link_type <- ifelse(q12$LinkTypeId == 3, "Duplicate", ifelse(q12$LinkTypeId == 1, "Related", as.cha

# A question can appear under both link types, count distinct questions per type and across both.
cat("Distinct questions that gave rise to at least one link, by type:\n")
```

```
## Distinct questions that gave rise to at least one link, by type:
```

```
aggregate(question_id ~ link_type, data = q12,
          FUN = function(x) length(unique(x)))
```

```
##   link_type question_id
## 1 Duplicate        7037
## 2   Related       29602
```

```r
cat("\nDistinct questions that gave rise to any related OR duplicate link:\n")
```

```
##
## Distinct questions that gave rise to any related OR duplicate link:
```

```r
length(unique(q12$question_id))
```

```
## [1] 30809
```

```r
# Responses received
cat("\nNumber of answers (responses) - summary, by link type:\n")
```

```
##
## Number of answers (responses) - summary, by link type:
```

```r
aggregate(n_answers ~ link_type, data = q12, FUN = summary)
```

```
##   link_type n_answers.Min. n_answers.1st Qu. n_answers.Median n_answers.Mean
## 1 Duplicate       0.000000          1.000000         2.000000       2.326844
## 2   Related       0.000000          1.000000         1.000000       1.799574
##   n_answers.3rd Qu. n_answers.Max.
## 1          3.000000      38.000000
## 2          2.000000     153.000000
```

```r
# Asker experience (reputation), by link type
cat("\nAsker reputation - summary, by link type:\n")
```

```
##
## Asker reputation - summary, by link type:
```

```r
aggregate(asker_reputation ~ link_type, data = q12,
          FUN = function(x) summary(x))
```

```
##   link_type asker_reputation.Min. asker_reputation.1st Qu.
## 1 Duplicate                 1.000                  121.000
## 2   Related                 1.000                  111.000
##   asker_reputation.Median asker_reputation.Mean asker_reputation.3rd Qu.
## 1                 403.000              3195.899                 1392.000
## 2                 343.500              3327.154                 1315.000
##   asker_reputation.Max.
## 1            304878.000
## 2            304878.000
```

```r
SELECT u.Reputation AS asker_reputation
FROM Posts p
JOIN Users u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 1
  AND p.OwnerUserId IS NOT NULL
  AND p.OwnerUserId != -1;
```

```r
cat("Reputation of askers whose questions spawned a link:\n")
```

```
## Reputation of askers whose questions spawned a link:
```

```r
summary(q12$asker_reputation)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##       1     111     355    3302    1336  304878     754
```

```r
cat("\nReputation of all questioners (baseline):\n")
```

```
##
## Reputation of all questioners (baseline):
```

```r
summary(q12_baseline$asker_reputation)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       1      25     131    1299     501  304878
```

#question 13

```sql
SELECT
  q.Id AS question_id,
  q.Title AS question_title,
  asker.DisplayName AS asker_name,
  asker.CreationDate AS asker_joined,
  asker.Location AS asker_location,
  q.CreationDate AS question_posted,
  a.Id AS accepted_answer_id,
  a.CreationDate AS answer_posted,
  answerer.DisplayName AS answerer_name
FROM Posts q
JOIN Posts a ON a.Id = q.AcceptedAnswerId
JOIN Users asker ON asker.Id    = q.OwnerUserId
JOIN Users answerer ON answerer.Id = a.OwnerUserId
WHERE q.PostTypeId = 1;
```

```r
nrow(q13)
```

```
## [1] 66213
```

```r
head(q13)
```

```
##   question_id
## 1           1
## 2           2
## 3           3
## 4           4
## 5           7
```

```
## 6          10
##                                                                    question_title
## 1                                                   Eliciting priors from experts
## 2                                                              What is normality?
## 3                 What are some valuable Statistical Analysis open source projects?
## 4                        Assessing the significance of differences in distributions
## 5                                             Locating freely available data samples
## 6 Under what conditions should Likert scales be used as ordinal or interval data?
##    asker_name            asker_joined              asker_location
## 1 csgillespie 2010-07-19T19:04:52.280   Newcastle, United Kingdom
## 2      A Lion 2010-07-19T19:09:32.157
## 3      grokus 2010-07-19T19:08:29.070               United States
## 4 Jay Stevens 2010-07-19T19:09:16.917       Jacksonville, FL, USA
## 5      EAMann 2010-07-19T19:11:57.393 Tualatin, OR, United States
## 6      A Lion 2010-07-19T19:09:32.157
##           question_posted accepted_answer_id           answer_posted
## 1 2010-07-19T19:12:12.510                 15 2010-07-19T19:19:46.160
## 2 2010-07-19T19:12:57.157                 59 2010-07-19T19:43:20.423
## 3 2010-07-19T19:13:28.577                  5 2010-07-19T19:14:43.050
## 4 2010-07-19T19:13:31.617                135 2010-07-19T21:36:12.850
## 5 2010-07-19T19:15:59.303                 18 2010-07-19T19:24:18.580
## 6 2010-07-19T19:17:47.537               1887 2010-08-19T10:00:00.370
##    answerer_name
## 1         Harlan
## 2 John L. Taylor
## 3    Jay Stevens
## 4 John L. Taylor
## 5 Stephen Turner
## 6            chl
```

#question 14

```sql
WITH accepted AS (
  SELECT
    a.Id AS answer_id,
    a.OwnerUserId AS user_id,
    a.CreationDate AS answer_date,
    a.ParentId AS question_id
  FROM Posts q
  JOIN Posts a ON a.Id = q.AcceptedAnswerId
  WHERE q.PostTypeId = 1
    AND a.OwnerUserId IS NOT NULL
    AND a.OwnerUserId != -1
),
top_users AS (
  SELECT user_id, COUNT(*) AS n_accepted
  FROM accepted
  GROUP BY user_id
  ORDER BY n_accepted DESC
  LIMIT 75
),
accepted_top AS (
  SELECT a.*
  FROM accepted a
```

```sql
  JOIN top_users t ON t.user_id = a.user_id
),
date_agg AS (
  SELECT
    user_id,
    MIN(answer_date) AS earliest_accepted,
    MAX(answer_date) AS latest_accepted
  FROM accepted_top
  GROUP BY user_id
),
tag_agg AS (
  SELECT
    at.user_id,
    GROUP_CONCAT(DISTINCT tp.Tag) AS tags
  FROM accepted_top at
  JOIN TagPosts tp ON tp.Id = at.question_id
  GROUP BY at.user_id
),
badge_agg AS (
  SELECT
    b.UserId AS user_id,
    COUNT(*) AS n_badges,
    GROUP_CONCAT(DISTINCT b.Name) AS badge_names
  FROM Badges b
  JOIN top_users t ON t.user_id = b.UserId
  GROUP BY b.UserId
)
SELECT
  u.DisplayName AS display_name,
  u.CreationDate AS account_created,
  u.Location AS location,
  t.n_accepted AS n_accepted_answers,
  COALESCE(ba.n_badges, 0) AS n_badges,
  ba.badge_names    AS badge_names,
  d.earliest_accepted,
  d.latest_accepted,
  ta.tags AS answered_question_tags
FROM top_users t
JOIN Users u ON u.Id = t.user_id
LEFT JOIN badge_agg ba ON ba.user_id = t.user_id
LEFT JOIN date_agg d ON d.user_id  = t.user_id
LEFT JOIN tag_agg ta ON ta.user_id = t.user_id
ORDER BY t.n_accepted DESC;
```

```r
nrow(q14)
```

```
## [1] 74
```

```r
head(q14[, c("display_name", "n_accepted_answers", "n_badges","earliest_accepted", "latest_accepted")])
```

```
##      display_name n_accepted_answers n_badges       earliest_accepted
## 1          Glen_b               2335     1605 2010-08-09T00:37:45.090
```

```
## 2          whuber                1781     1942 2010-08-19T13:07:22.083
## 3             EdM                1246      318 2013-08-13T20:01:37.963
## 4           gunes                1119      129 2018-09-09T17:27:12.530
## 5             Tim                1004      717 2014-10-31T11:05:38.017
## 6 Stephan Kolassa                 985      638 2010-09-23T20:42:45.613
##           latest_accepted
## 1 2023-02-23T22:11:02.110
## 2 2023-03-05T00:12:24.030
## 3 2023-03-02T21:10:15.797
## 4 2023-01-22T13:32:10.323
## 5 2023-02-25T07:53:52.917
## 6 2023-03-02T07:47:57.410
```

```sql
WITH tag_counts AS (
  SELECT Id AS qid, COUNT(*) AS n_tags
  FROM TagPosts
  GROUP BY Id
),
answer_counts AS (
  SELECT ParentId AS qid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
),
comment_counts AS (
  SELECT PostId AS qid, COUNT(*) AS n
  FROM Comments
  GROUP BY PostId
)
SELECT
  q.Id AS question_id,
  COALESCE(t.n_tags, 0) AS n_tags,
  LENGTH(q.Body) AS body_length,
  COALESCE(a.n, 0) AS n_answers,
  COALESCE(c.n, 0) AS n_comments,
  COALESCE(a.n, 0) + COALESCE(c.n, 0) AS n_responses
FROM Posts q
LEFT JOIN tag_counts     t ON t.qid = q.Id
LEFT JOIN answer_counts  a ON a.qid = q.Id
LEFT JOIN comment_counts c ON c.qid = q.Id
WHERE q.PostTypeId = 1;
```

```r
summary(q15[, c("n_tags", "body_length", "n_answers", "n_comments", "n_responses")])
```

```
##      n_tags       body_length      n_answers          n_comments
##  Min.   :1.000   Min.   :   40   Min.   :  0.0000   Min.   : 0.000
##  1st Qu.:2.000   1st Qu.:  563   1st Qu.:  0.0000   1st Qu.: 0.000
##  Median :3.000   Median :  938   Median :  1.0000   Median : 1.000
##  Mean   :3.085   Mean   : 1300   Mean   :  0.9685   Mean   : 2.171
##  3rd Qu.:4.000   3rd Qu.: 1567   3rd Qu.:  1.0000   3rd Qu.: 3.000
##  Max.   :5.000   Max.   :69910   Max.   :153.0000   Max.   :54.000
##   n_responses
##  Min.   :  0.00
```

```
##  1st Qu.:  1.00
##  Median :  2.00
##  Mean   :  3.14
##  3rd Qu.:  4.00
##  Max.   :154.00
```

```r
# Spearman (rank-based) is appropriate given heavy right skew in all variables
cor(
  q15[, c("n_tags", "body_length", "n_answers", "n_comments", "n_responses")], method = "spearman"
)
```

```
##                   n_tags   body_length   n_answers   n_comments   n_responses
## n_tags       1.000000000  0.2139236843 -0.04828000  0.008046655 -0.0112751019
## body_length  0.213923684  1.0000000000 -0.09600000  0.040911566  0.0001178435
## n_answers   -0.048280004 -0.0960000003  1.00000000 -0.077347144  0.3038959438
## n_comments   0.008046655  0.0409115655 -0.07734714  1.000000000  0.9007316292
## n_responses -0.011275102  0.0001178435  0.30389594  0.900731629  1.0000000000
```

```r
# Mean responses and length by number of tags
aggregate(cbind(body_length, n_answers, n_comments, n_responses) ~ n_tags,
          data = q15, FUN = mean)
```

```
##   n_tags body_length n_answers n_comments n_responses
## 1      1    914.0764 1.0746417   2.016019    3.090661
## 2      2   1094.0551 1.0065344   2.147591    3.154126
## 3      3   1266.5241 0.9536984   2.160834    3.114533
## 4      4   1451.6878 0.9318366   2.202294    3.134130
## 5      5   1722.1678 0.9199636   2.283591    3.203555
```

```r
# Length deciles vs response counts
q15$length_decile <- cut(q15$body_length, breaks = quantile(q15$body_length, probs = 0:10/10,na.rm = TRU
aggregate(cbind(n_answers, n_comments, n_responses) ~ length_decile,
          data = q15, FUN = mean)
```

```
##    length_decile n_answers n_comments n_responses
## 1              1 1.1957042   1.883280    3.078985
## 2              2 1.0647676   1.953814    3.018582
## 3              3 1.0180793   1.979636    2.997716
## 4              4 0.9818548   2.023800    3.005655
## 5              5 0.9533664   2.075614    3.028981
## 6              6 0.9284809   2.142112    3.070593
## 7              7 0.9062546   2.204144    3.110398
## 8              8 0.8910925   2.262671    3.153763
## 9              9 0.8774310   2.390927    3.268358
## 10            10 0.8669507   2.798443    3.665393
```

```r
library(ggplot2)
# Mean responses by tag count
agg_tags <- aggregate(cbind(n_answers, n_comments, n_responses) ~ n_tags, data = q15, FUN = mean)
agg_tags
```

```
##   n_tags n_answers n_comments n_responses
## 1      1 1.0746417   2.016019    3.090661
## 2      2 1.0065344   2.147591    3.154126
## 3      3 0.9536984   2.160834    3.114533
## 4      4 0.9318366   2.202294    3.134130
## 5      5 0.9199636   2.283591    3.203555
```

#Question 16

```sql
SELECT * FROM VoteTypeMap;
```

Table 4: Displaying records 1 - 10

id	value
1	AcceptedByOriginator
2	UpMod (AKA upvote)
3	DownMod (AKA downvote)
4	Offensive
5	Favorite (UserId will also be populated)
6	Close (effective 2013-06-25: Close votes are only stored in table: PostHistory)
7	Reopen
8	BountyStart (UserId and BountyAmount will also be populated)
9	BountyClose (BountyAmount will also be populated)
10	Deletion

```sql
WITH bounty_agg AS (
  SELECT
    PostId AS qid,
    COUNT(*) AS n_bounties,
    SUM(BountyAmount) AS total_bounty,
    MAX(BountyAmount) AS max_bounty
  FROM Votes
  WHERE VoteTypeId = 8
  GROUP BY PostId
),
answer_agg AS (
  SELECT
    ParentId AS qid,
    COUNT(*) AS n_answers,
    MIN(CreationDate) AS first_answer_date
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
)
SELECT
  q.Id AS question_id,
  q.CreationDate AS question_date,
  CASE WHEN b.qid IS NOT NULL THEN 1 ELSE 0 END AS has_bounty,
  COALESCE(b.total_bounty, 0) AS total_bounty,
  COALESCE(b.max_bounty, 0) AS max_bounty,
  COALESCE(a.n_answers, 0) AS n_answers,
```

```
  (julianday(a.first_answer_date) - julianday(q.CreationDate)) * 24 AS hours_to_first,
  CASE WHEN q.AcceptedAnswerId IS NOT NULL THEN 1 ELSE 0 END AS accepted
FROM Posts q
LEFT JOIN bounty_agg b ON b.qid = q.Id
LEFT JOIN answer_agg a ON a.qid = q.Id
WHERE q.PostTypeId = 1;
```

```
# Effect of bounty PRESENCE
cat("Counts:\n")
```

```
## Counts:
```

```
table(has_bounty = q16$has_bounty)
```

```
## has_bounty
##      0      1
## 198428   5942
```

```
cat("\nNumber of answers by bounty presence:\n")
```

```
##
## Number of answers by bounty presence:
```

```
aggregate(n_answers ~ has_bounty, data = q16, FUN = function(x) c(mean = mean(x), median = median(x)))
```

```
##   has_bounty n_answers.mean n_answers.median
## 1          0      0.9505161        1.0000000
## 2          1      1.5683272        1.0000000
```

```
cat("\nHours to first answer by bounty presence (median, mean):\n")
```

```
##
## Hours to first answer by bounty presence (median, mean):
```

```
aggregate(hours_to_first ~ has_bounty, data = q16, FUN = function(x) c(median = median(x, na.rm = TRUE)
```

```
##   has_bounty hours_to_first.median hours_to_first.mean
## 1          0              2.836172         1291.994466
## 2          1            133.952516         1682.568679
```

```
cat("\nAcceptance rate by bounty presence:\n")
```

```
##
## Acceptance rate by bounty presence:
```

```
aggregate(accepted ~ has_bounty, data = q16, FUN = mean)
```

```
##   has_bounty accepted
## 1          0        1
## 2          1        1
```

```r
bountied <- subset(q16, has_bounty == 1)

cat("Acceptance rate within bountied questions:\n")
```

```
## Acceptance rate within bountied questions:
```

```r
mean(bountied$accepted)   # likely 1.0 or very close
```

```
## [1] 1
```

```r
cat("\nSpearman correlations within bountied questions (acceptance omitted - no variance):\n")
```

```
##
## Spearman correlations within bountied questions (acceptance omitted - no variance):
```

```r
cor(
  bountied[, c("total_bounty", "max_bounty", "n_answers", "hours_to_first")],
  use = "pairwise.complete.obs",
  method = "spearman"
)
```

```
##                total_bounty  max_bounty  n_answers hours_to_first
## total_bounty     1.00000000  0.98789642  0.1284373    -0.09054793
## max_bounty       0.98789642  1.00000000  0.1217193    -0.09431316
## n_answers        0.12843735  0.12171935  1.0000000    -0.27979726
## hours_to_first  -0.09054793 -0.09431316 -0.2797973     1.00000000
```

```r
bountied$bounty_bin <- cut(bountied$total_bounty,
                           breaks = c(0, 50, 100, 200, 500, Inf),
                           labels = c("1-50", "51-100", "101-200",
                                      "201-500", "500+"),
                           include.lowest = TRUE)

cat("\nOutcomes by bounty amount bin:\n")
```

```
##
## Outcomes by bounty amount bin:
```

```r
aggregate(cbind(n_answers, hours_to_first, accepted) ~ bounty_bin, data = bountied, FUN = function(x) c
```

```
##   bounty_bin n_answers.mean n_answers.median hours_to_first.mean
## 1       1-50       1.732662         1.000000          1743.66182
## 2     51-100       2.059267         2.000000          1210.89857
## 3    101-200       2.261981         2.000000          2089.31648
## 4    201-500       2.893204         2.000000          2559.29048
## 5       500+       2.384615         2.000000          1333.02762
##   hours_to_first.median accepted.mean accepted.median
## 1             144.19353             1               1
## 2              95.97806             1               1
## 3             119.71102             1               1
## 4             140.51400             1               1
## 5              66.11710             1               1
```